

Linked lists

COMPLETE LEARNING GUIDE

What You'll Learn

A linked list is a linear data structure where each element (node) holds its data plus a reference to the next node, trading array's $O(1)$ random access for $O(1)$ insertion and deletion at any known position — a fundamental structural trade-off interviewers expect you to articulate clearly. This guide covers building a singly linked list from scratch, traversal, insertion, and deletion, the classic iterative in-place reversal algorithm, Floyd's cycle-detection algorithm (the "tortoise and hare"), a brief look at doubly and circular linked lists, and the broader family of fast/slow pointer techniques that linked-list problems rely on constantly. Linked list manipulation is one of the most reliable indicators interviewers use to assess pointer-handling discipline, since a single misplaced reference update can silently corrupt the entire list or create an infinite loop.

1. Node Structure and Basic Traversal

A singly linked list node holds a value and a reference to the next node (null for the last node), and the list itself typically only needs to track a reference to the head node — every other node is reached by following next references one at a time. Traversal is always $O(n)$, since there is no way to "skip ahead" in a linked list the way array indexing allows; this is the fundamental trade-off versus an array, and it's why algorithms on linked lists lean heavily on maintaining multiple pointers (current, previous, fast, slow) rather than indexing.

JAVA

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}

// Traversal: O(n) time, O(1) space
public static void printList(ListNode head) {
    ListNode current = head;
    while (current != null) {
        System.out.print(current.val + " -> ");
        current = current.next;
    }
    System.out.println("null");
}
```

2. Insertion and Deletion

Inserting at the head is $O(1)$ — create a new node whose next points at the current head, then update the list's head reference to the new node. Inserting after a given node (with a reference already in hand) is also $O(1)$ — relink two next pointers. Inserting at an arbitrary position by index, or at the tail without a maintained tail pointer, requires first traversing to find that position, making it $O(n)$ overall even though the actual insertion step is $O(1)$ — this is the nuance interviewers probe when asking "what's the complexity of linked list insertion," since the honest answer is " $O(1)$ if you already have the position, $O(n)$ if you have to find it first."

JAVA

```
// Delete the node with a given value --  $O(n)$  (must find it),  $O(1)$  extra space
public static ListNode deleteValue(ListNode head, int value) {
    ListNode dummy = new ListNode(0); // dummy node simplifies head-deletion
    dummy.next = head;
    ListNode prev = dummy;

    while (prev.next != null) {
        if (prev.next.val == value) {
            prev.next = prev.next.next; // unlink the matching node
            break;
        }
        prev = prev.next;
    }
    return dummy.next;
}
```

3. Reversing a Linked List Iteratively

Reversing a singly linked list in-place is one of the most commonly asked linked-list problems, and the standard iterative solution walks the list once, re-pointing each node's next to its predecessor instead of its successor, using three tracked pointers (previous, current, next) to avoid losing the rest of the list once a link is rewritten. It runs in $O(n)$ time with $O(1)$ extra space, versus a recursive version which is also $O(n)$ time but $O(n)$ space due to the call stack — this space difference is worth stating explicitly if asked to compare the two approaches.

JAVA

```
// Iterative reversal --  $O(n)$  time,  $O(1)$  space
public static ListNode reverse(ListNode head) {
    ListNode previous = null;
    ListNode current = head;

    while (current != null) {
        ListNode next = current.next; // save the rest of the list before
        current.next = previous; // reverse this node's pointer
        previous = current; // advance previous
        current = next; // advance current
    }
    return previous; // previous is now the new head
}
```

4. Floyd's Cycle Detection (Fast/Slow Pointers)

Detecting whether a linked list contains a cycle without extra memory is solved elegantly by Floyd's algorithm: two pointers start at the head, one (slow) advancing one node per step and the other (fast) advancing two nodes per step — if there is no cycle, fast reaches null and the algorithm terminates; if there is a cycle, fast will eventually "lap" slow and the two pointers will point at the identical node, proving a cycle exists. This is provably correct because the gap between fast and slow shrinks by exactly one node per iteration once both are inside the cycle, guaranteeing they meet within at most the cycle's length in iterations.

JAVA

```
// Floyd's cycle detection -- O(n) time, O(1) space (vs O(n) space for a HashSet-based approach)
public static boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;          // moves 1 step
        fast = fast.next.next;     // moves 2 steps
        if (slow == fast) return true; // they met -- a cycle exists
    }
    return false; // fast reached the end -- no cycle
}
```

5. Doubly and Circular Linked Lists

A doubly linked list adds a prev reference alongside next on every node, enabling $O(1)$ backward traversal and $O(1)$ deletion of a known node without needing a separate reference to its predecessor (unlike a singly linked list, where deleting a node requires already having a reference to the node before it) — the trade-off is extra memory per node and more pointer updates to keep consistent on every mutation. A circular linked list has its last node's next point back to the head instead of null, useful for round-robin scheduling or any repeating-cycle use case, but it requires careful termination logic in traversal (checking against the starting node, not against null) to avoid an infinite loop.

6. The Fast/Slow Pointer Pattern Beyond Cycle Detection

The same two-speed pointer idea that detects cycles also solves several other classic linked-list problems in a single pass with $O(1)$ extra space: finding the middle node (when fast reaches the end, slow is exactly at the midpoint, since it has traveled half as far), and finding the n th node from the end (advance one pointer n steps first, then move both pointers together — when the lead pointer reaches the end, the trailing pointer is exactly n nodes from the end). Recognizing "this needs two pointers moving at different rates" is one of the highest-leverage pattern-recognition skills for linked-list interview questions.

Common Pitfalls

- Losing the rest of the list by overwriting `current.next` before saving a reference to it, breaking the reversal algorithm's ability to continue traversing.
- Forgetting to update the head reference after inserting or deleting at the front of the list, leaving the caller with a stale head pointer.
- Assuming linked list insertion/deletion is always $O(1)$ without accounting for the $O(n)$ traversal needed to reach an arbitrary position first.
- Using a HashSet to detect cycles ($O(n)$ space) when Floyd's algorithm solves it in $O(1)$ space — a common missed optimization.
- Off-by-one errors in the "nth node from the end" two-pointer pattern — miscounting how many steps to advance the lead pointer before starting to move both.
- Not handling the empty list (`head == null`) or single-node list as edge cases, causing a `NullPointerException` in traversal logic.
- Forgetting to null-terminate the new tail after reversal, if the original head (now the new tail) still points at the old second node.

Key Takeaways

- Linked lists trade $O(1)$ random access (which arrays have) for $O(1)$ insertion/deletion at a known position — traversal to find that position is still $O(n)$.
- Iterative reversal uses three pointers (previous, current, next) and runs in $O(n)$ time, $O(1)$ space — strictly better space complexity than a recursive version.
- Floyd's cycle detection (fast/slow pointers) finds a cycle in $O(n)$ time and $O(1)$ space, beating a HashSet-based approach's $O(n)$ space.
- The fast/slow pointer pattern also finds the middle node and the nth-from-end node in a single pass with $O(1)$ extra space.
- A dummy/sentinel node before the head simplifies edge cases (deleting the head itself) in insertion/deletion code.
- Doubly linked lists trade extra per-node memory for $O(1)$ backward traversal and simpler deletion given only the node itself.

Learn More

GeeksforGeeks — Linked List Data Structure

<https://www.geeksforgeeks.org/dsa/linked-list-data-structure/>

GeeksforGeeks — Detect Loop in a Linked List (Floyd's Algorithm)

<https://www.geeksforgeeks.org/dsa/detect-loop-in-a-linked-list/>

GeeksforGeeks — Reverse a Linked List

<https://www.geeksforgeeks.org/dsa/reverse-a-linked-list/>